LENGUAJE DE PROGRAMACIÓN II

Seguridad frontend en PharmaSoft

Actividad práctica · Sesión 11 · Unidad II

Login, sesión con vencimiento, interceptores, guards y permisos. Evidencia de 16 pruebas con Angular, Spring Boot y Oracle SYSVENTAS.

Proyecto existente: Tinoco_LP2_AP9_Codigo / pharma-ap9

Rama: feature/seguridad-frontend-tinoco

Fecha: 22 de octubre de 2026

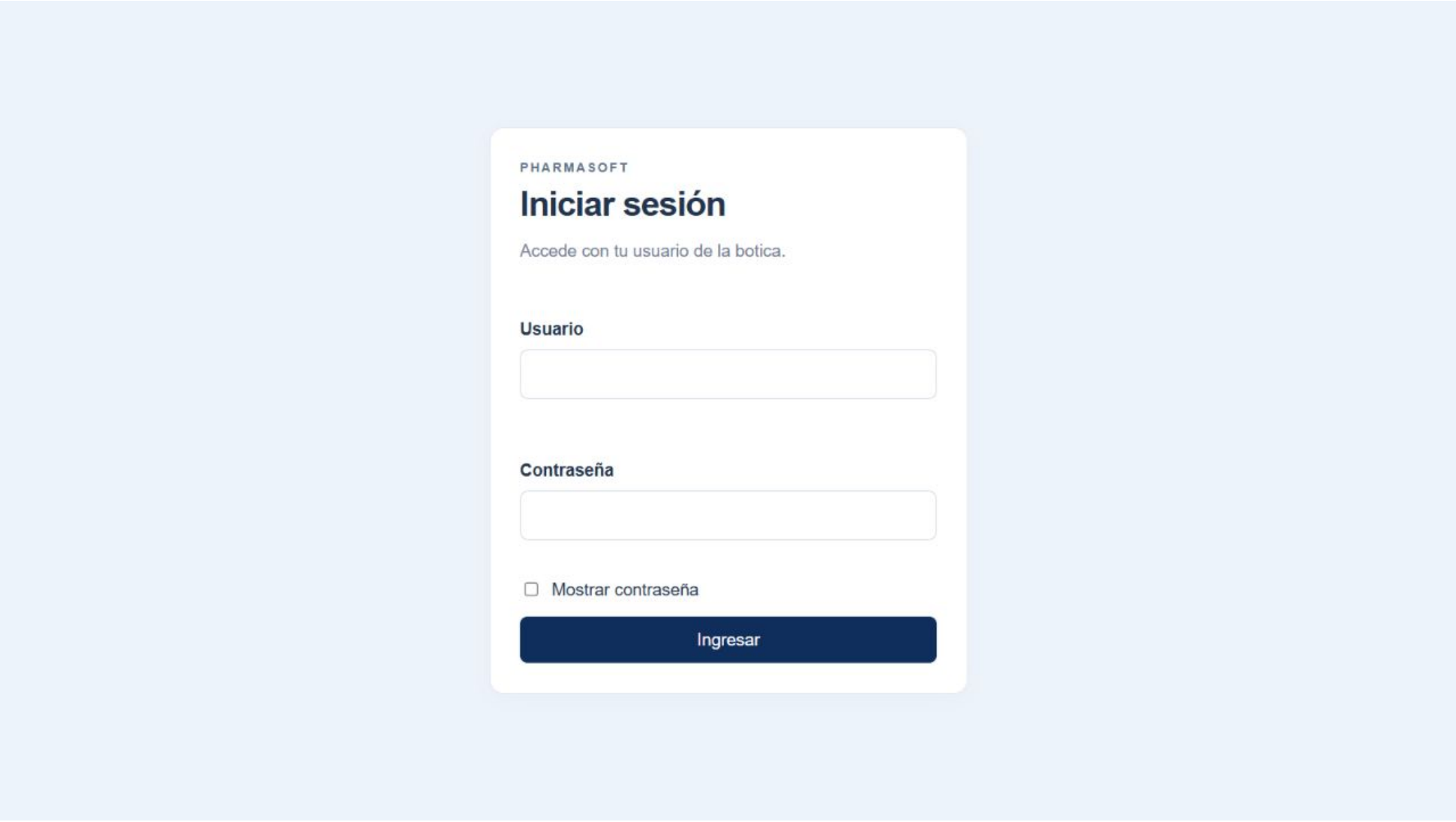
16 / 16 casos ejecutados con backend y Oracle reales

Cinco perfiles: admin, vendedor, almacén, supervisor e inactivo.

Build Angular correcto · 1 prueba automatizada aprobada · backend compilado.

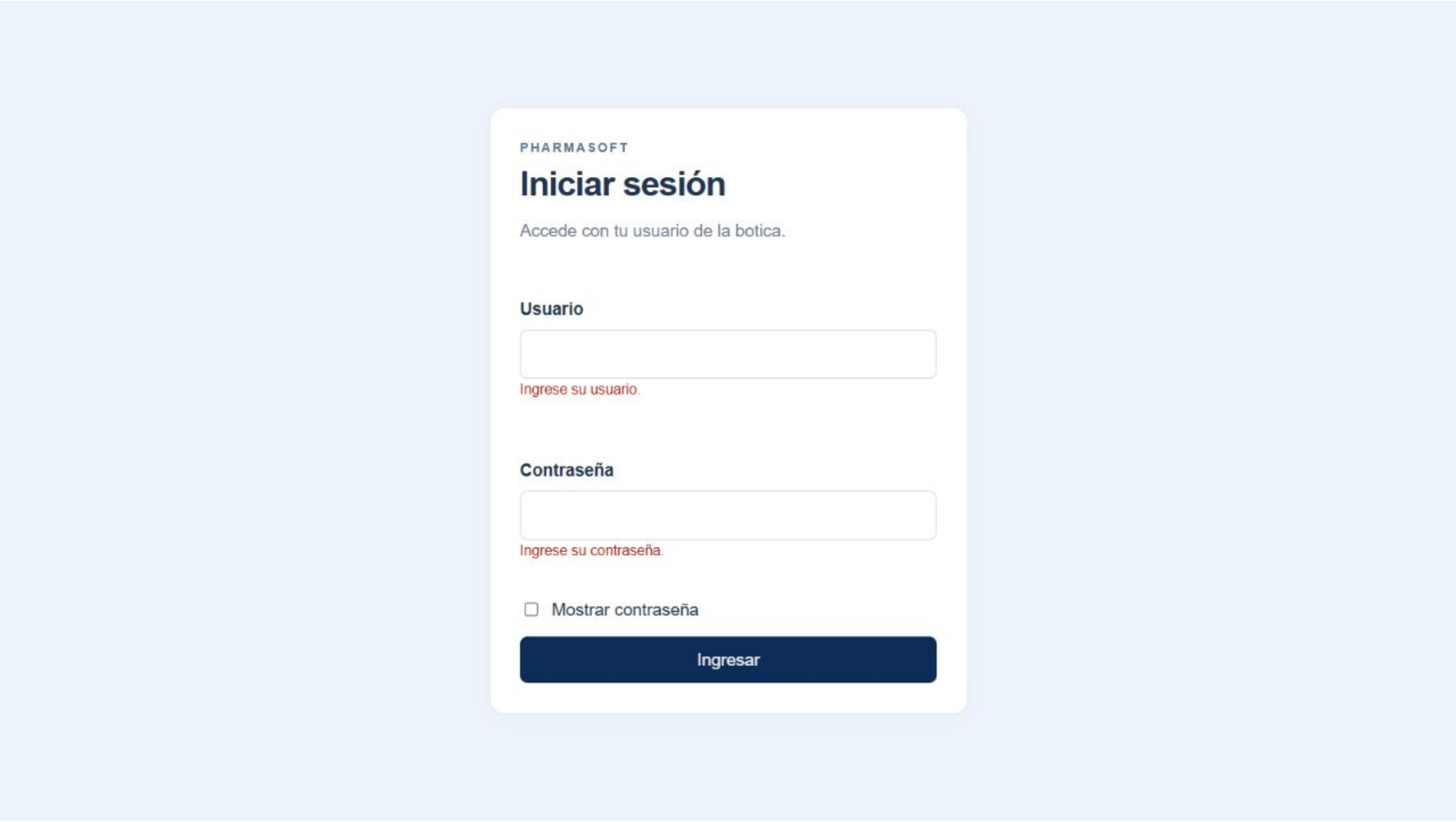
01 Redirección	OK	09 Solo lectura	OK
02 Validación	OK	10 Ruta denegada	OK
03 Clave errónea	OK	11 Invitado	OK
04 Inactivo	OK	12 Logout	OK
05 Vendedor	OK	13 Almacén	OK
06 Bearer	OK	14 Supervisor	OK
07 Almacenamiento	OK	15 Token alterado	OK
08 F5	OK	16 Vencimiento	OK

PRUEBA 01 / 16 · Sin sesión: ruta /ventas



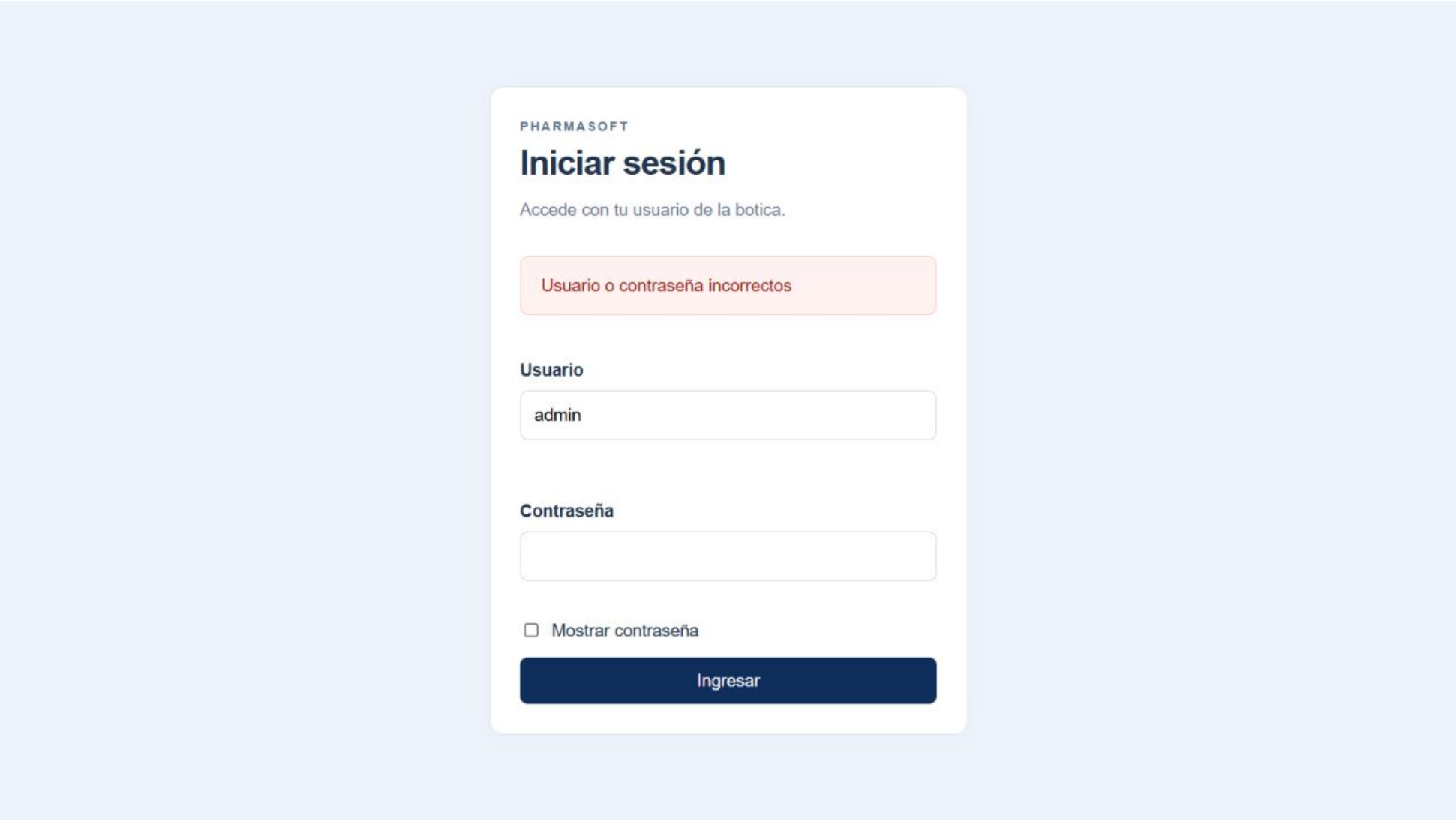
Resultado: La SPA envió al login con returnUrl=%2Fventas.

PRUEBA 02 / 16 · Formulario vacío



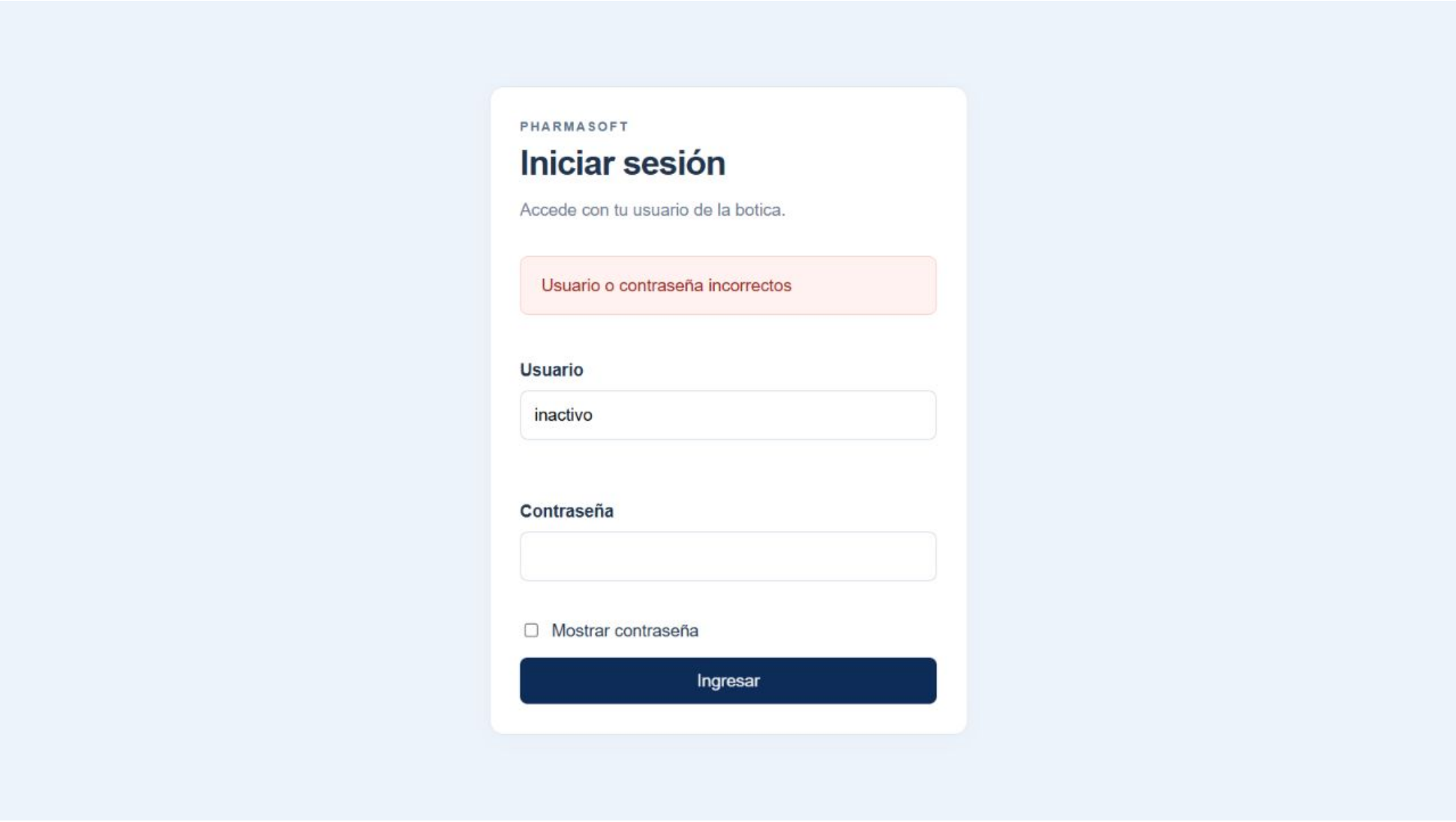
Resultado: Se mostraron los dos mensajes de validación; no se envió login.

PRUEBA 03 / 16 · Contraseña incorrecta



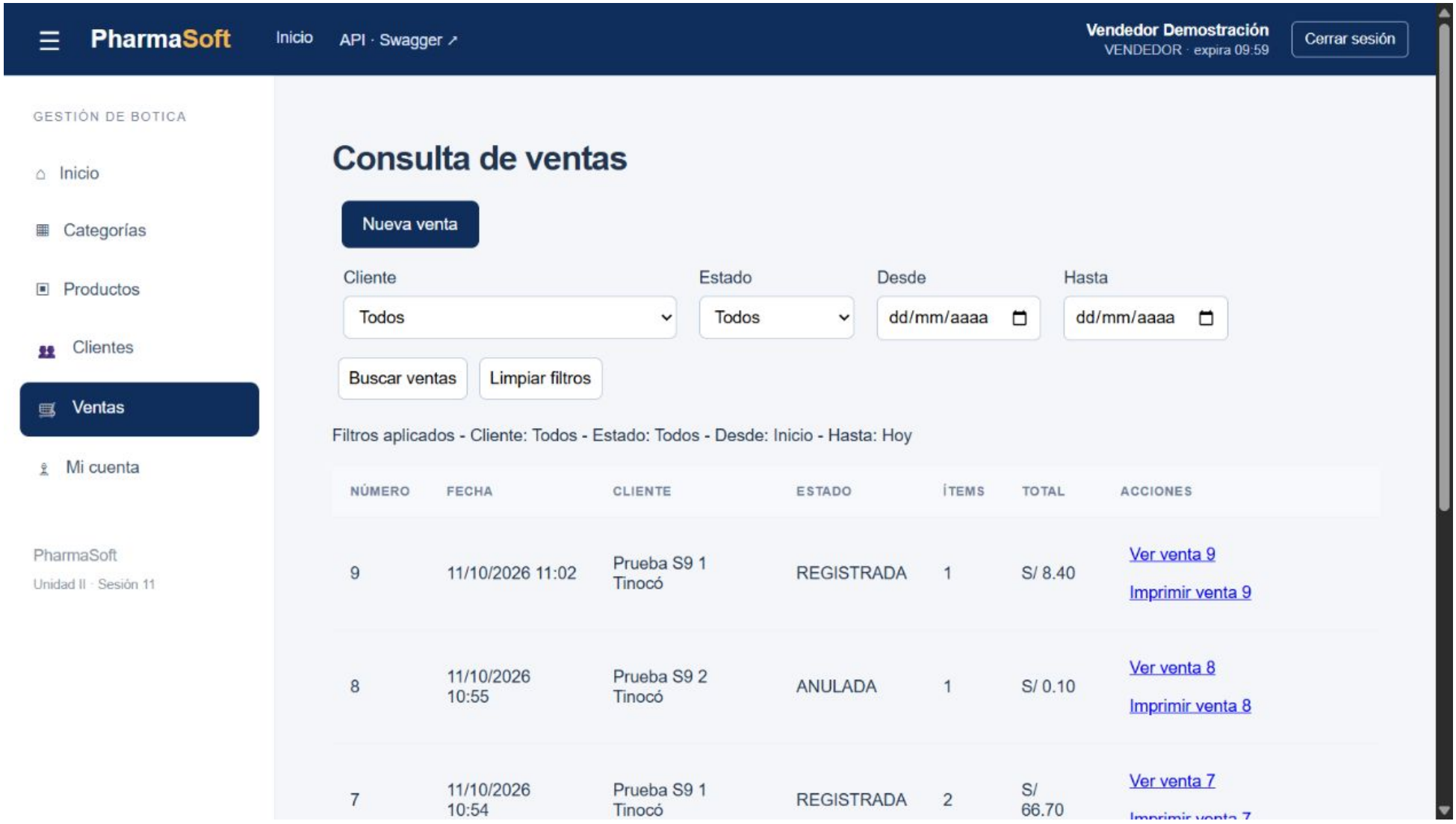
Resultado: El backend respondió 401 y el formulario mostró el mensaje genérico.

PRUEBA 04 / 16 · Cuenta inactiva



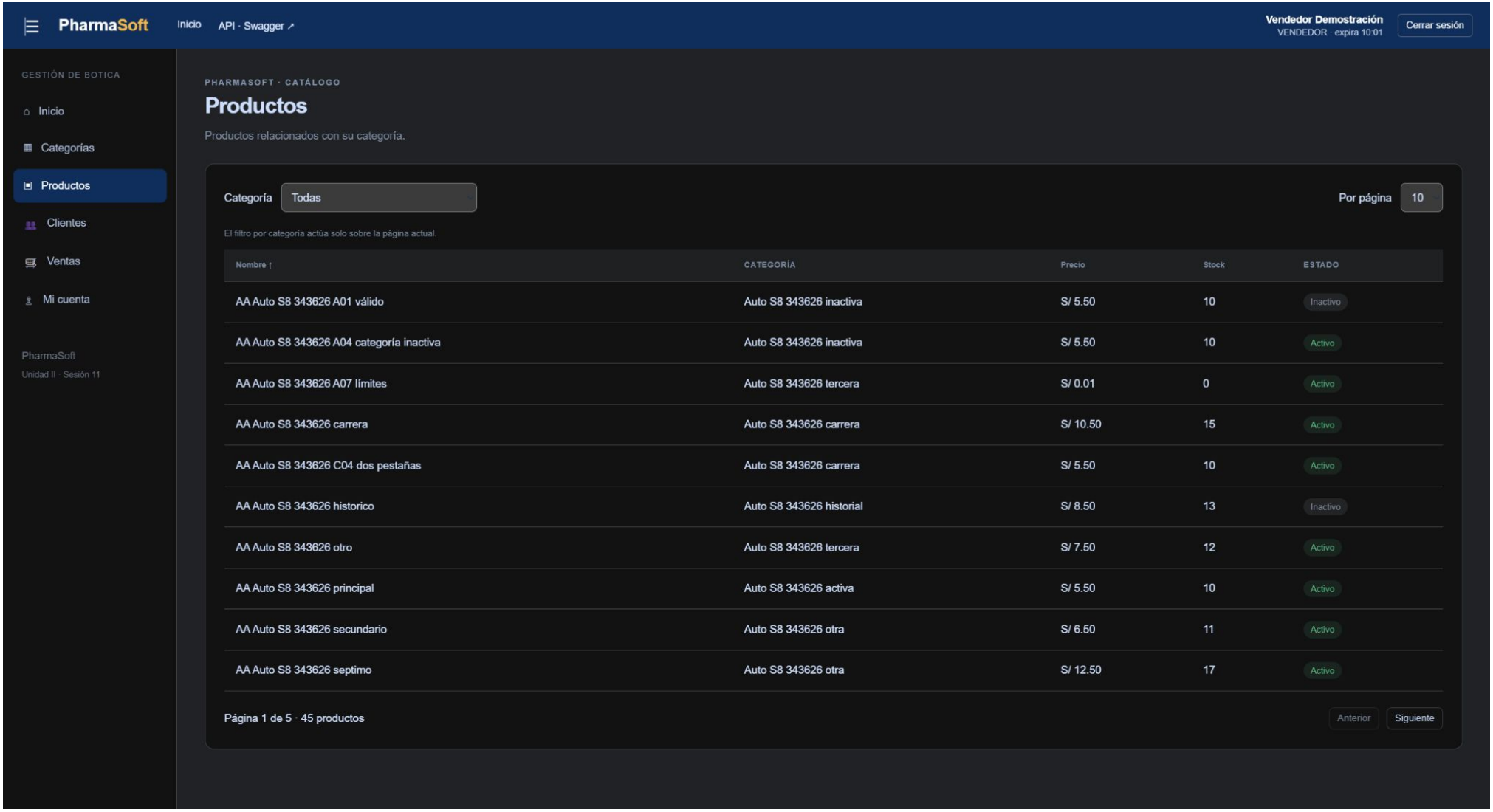
Resultado: El backend respondió 401 con el mismo mensaje genérico.

PRUEBA 05 / 16 · Ingreso de VENDEDOR



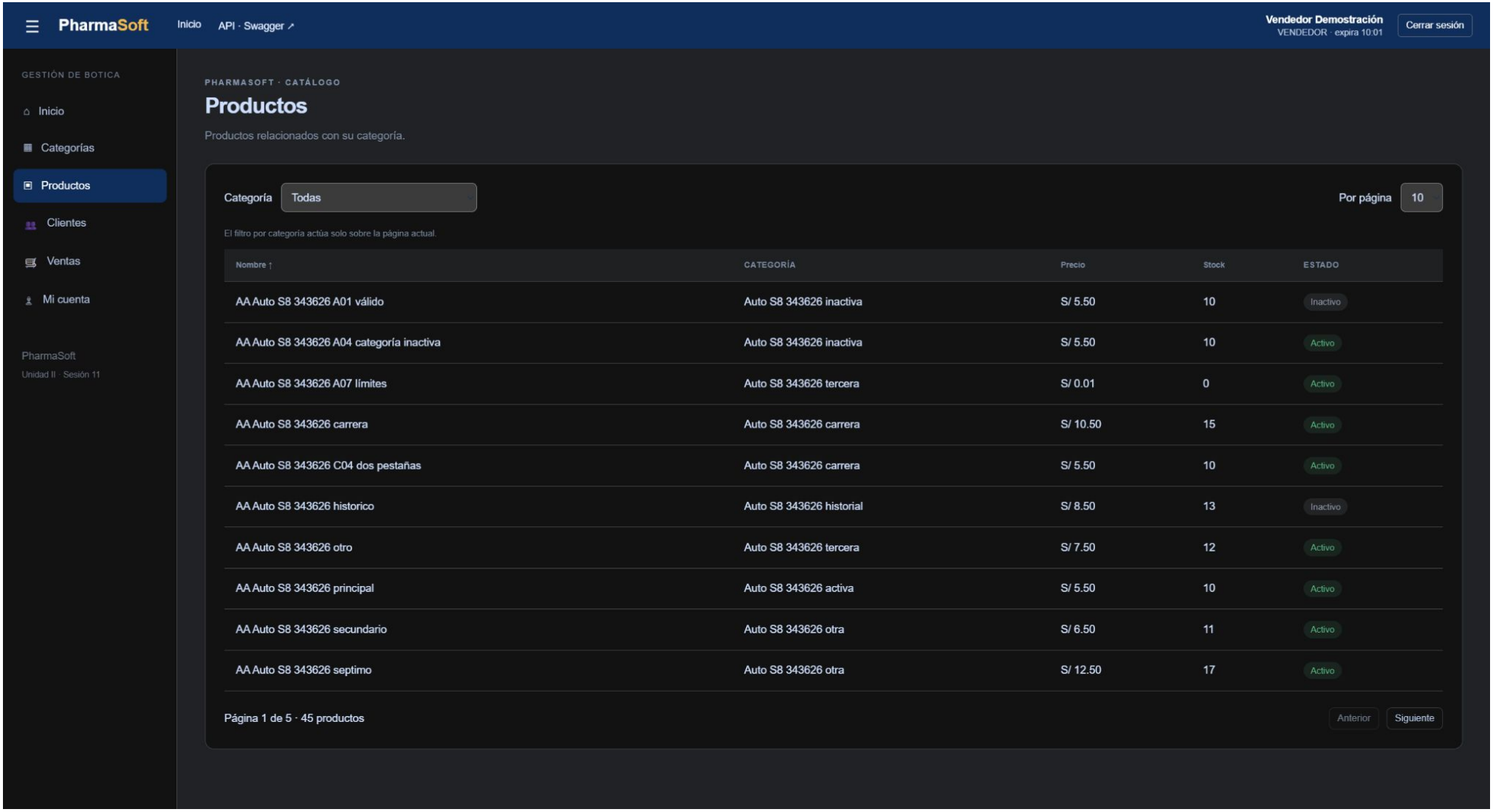
Resultado: Volvió a /ventas; el encabezado mostró nombre, rol y vencimiento.

PRUEBA 06 / 16 · GET /productos con Bearer



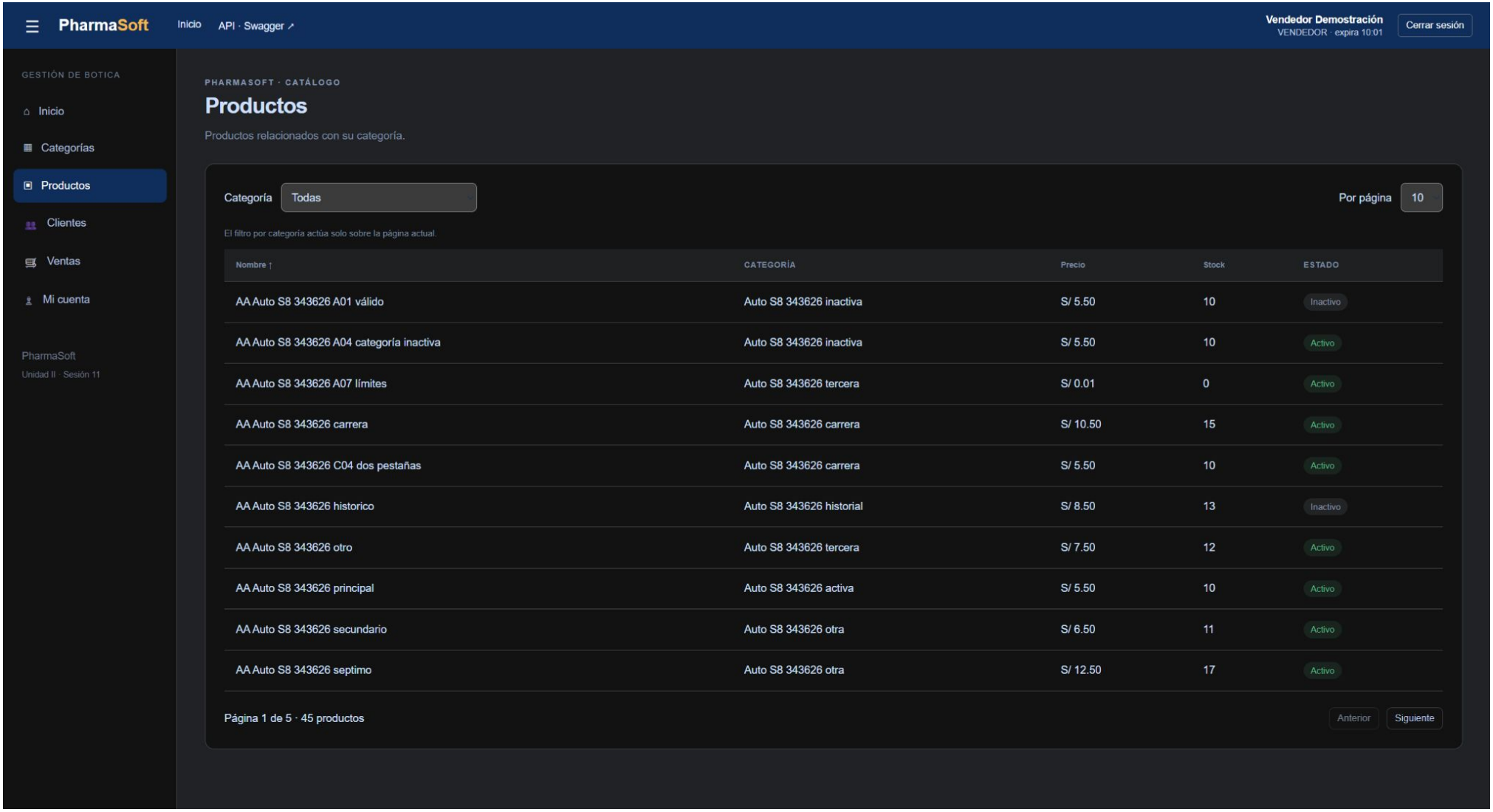
Resultado: Tráfico real: POST /auth/login sin Bearer; GET /productos 200 con Bearer. Token omitido.

PRUEBA 07 / 16 · Sesión en sessionStorage



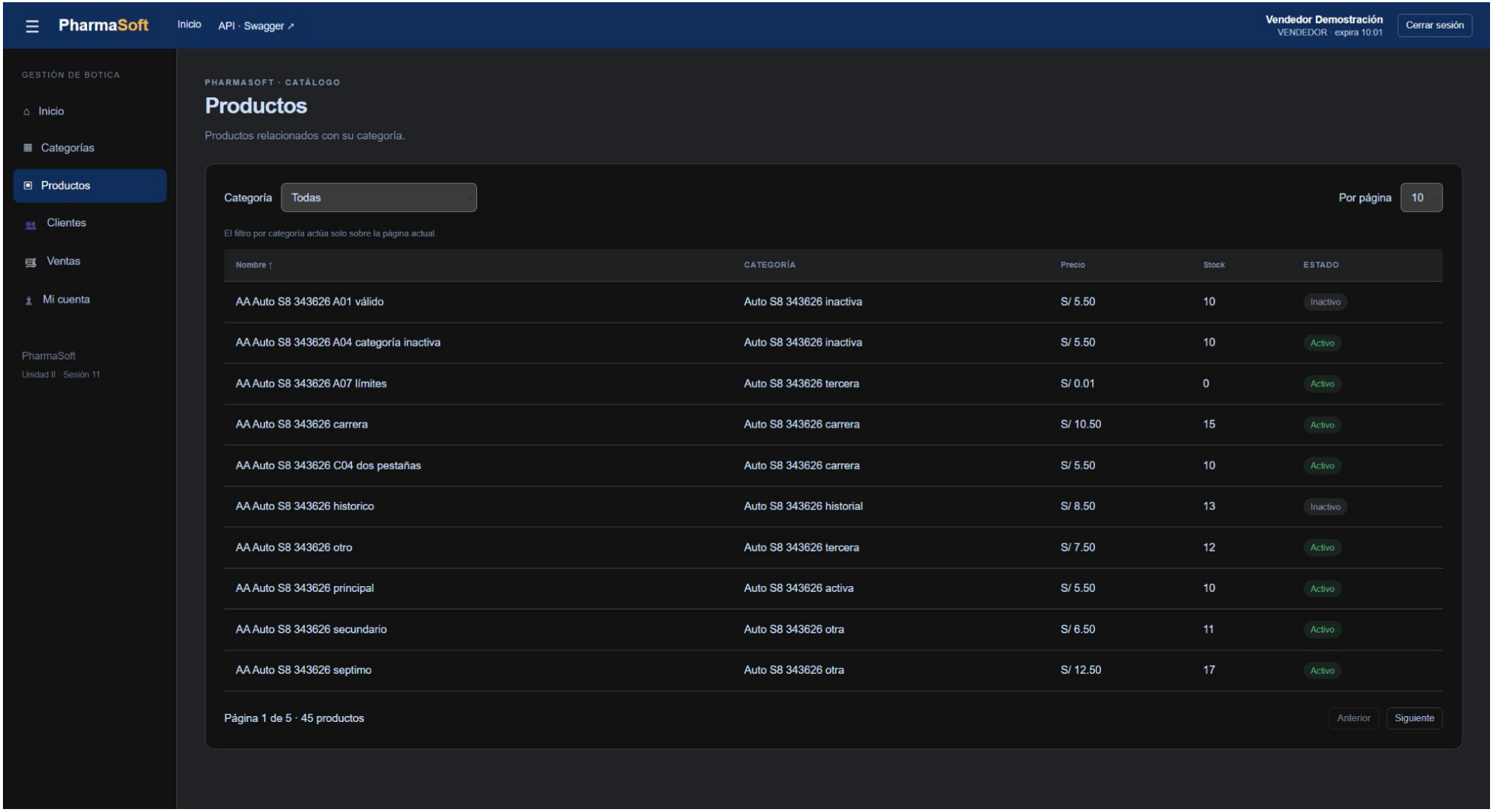
Resultado: Clave pharmasoft.sesion con token, expiraEn y permisos. Valor del token omitido.

PRUEBA 08 / 16 · F5 en Productos



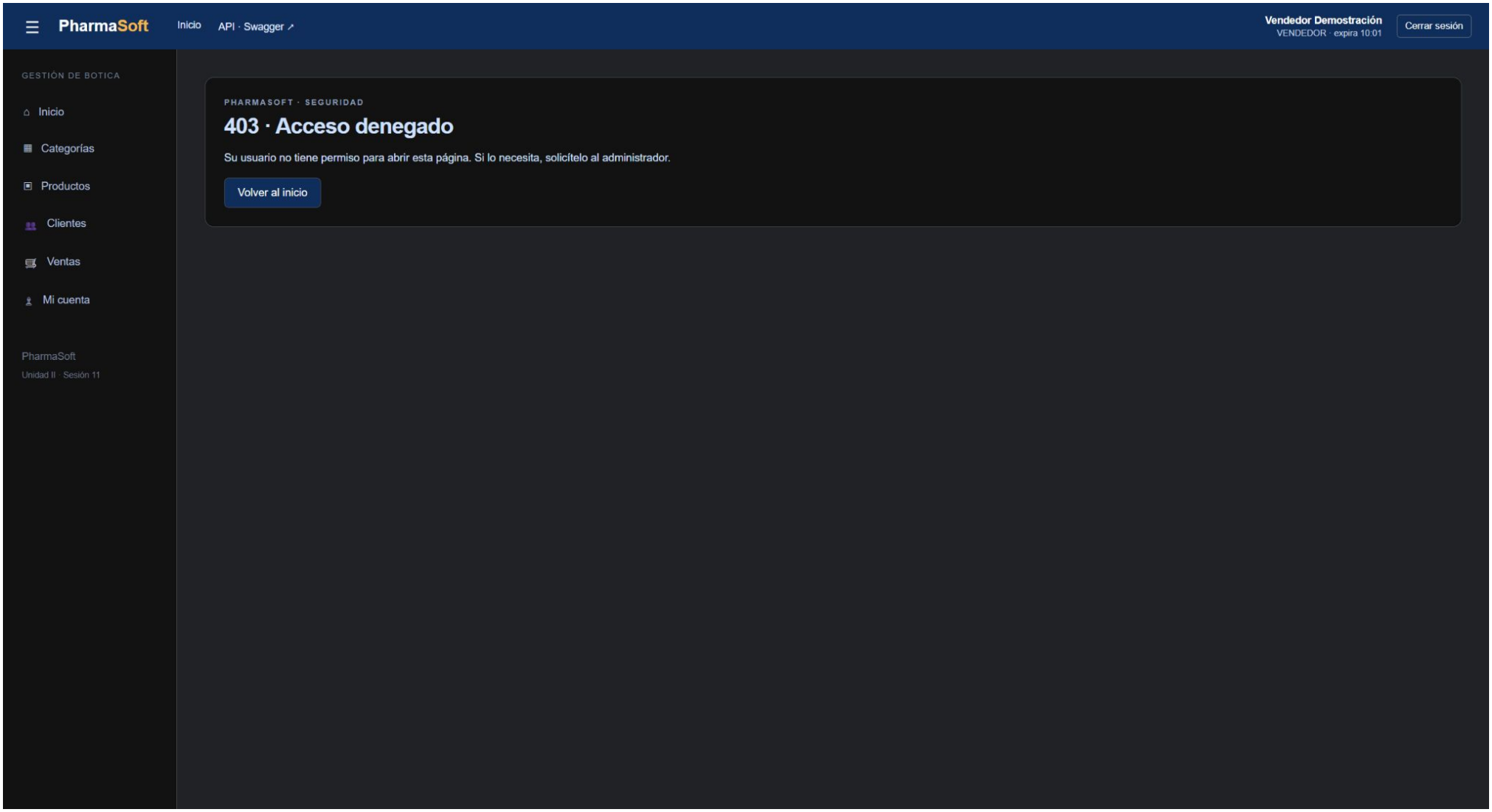
Resultado: La página siguió autenticada después de la recarga.

PRUEBA 09 / 16 · VENDEDOR sin edición



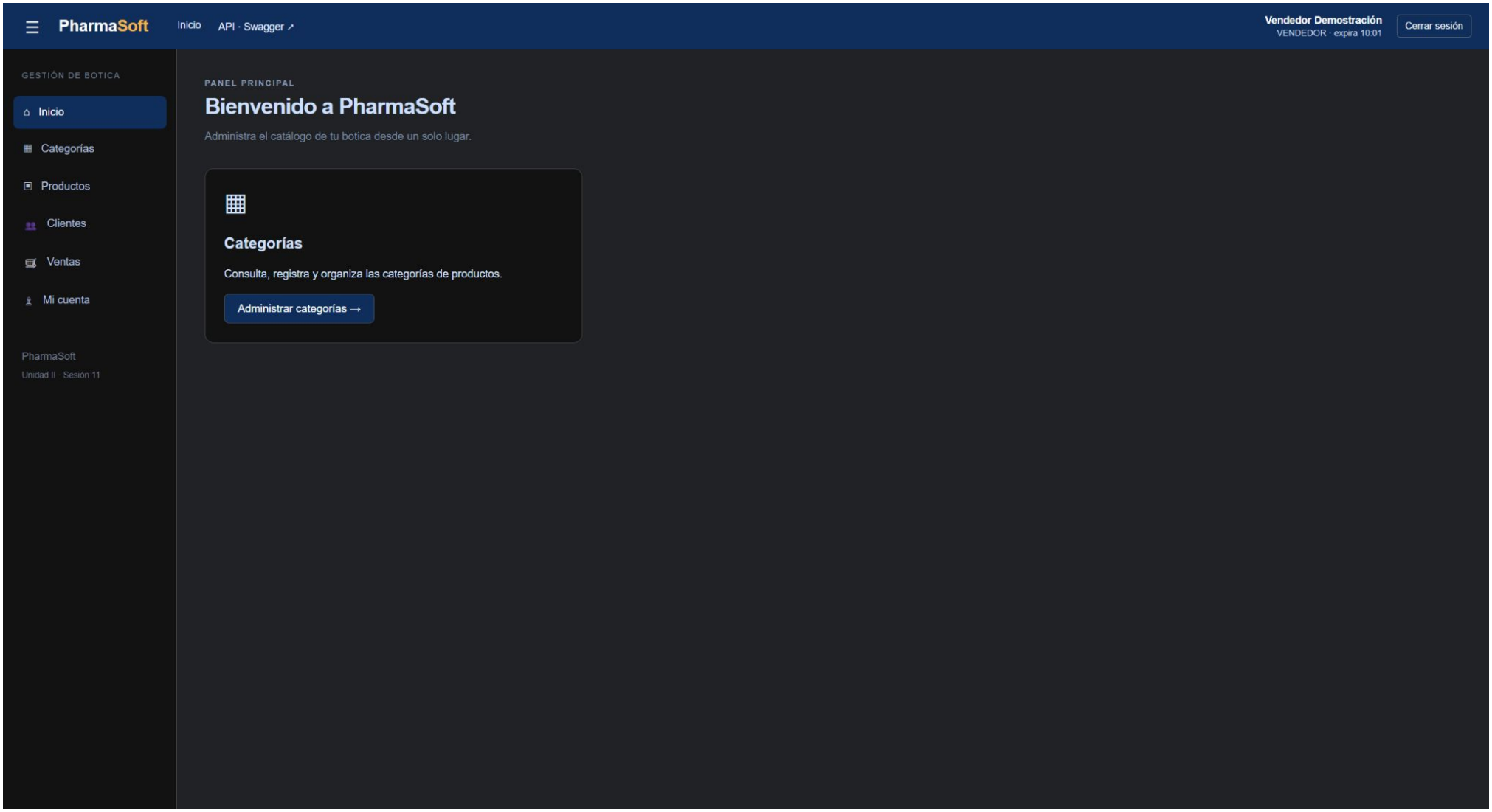
Resultado: Productos no mostró Nuevo producto ni columna Acciones.

PRUEBA 10 / 16 · VENDEDOR en /productos/nuevo



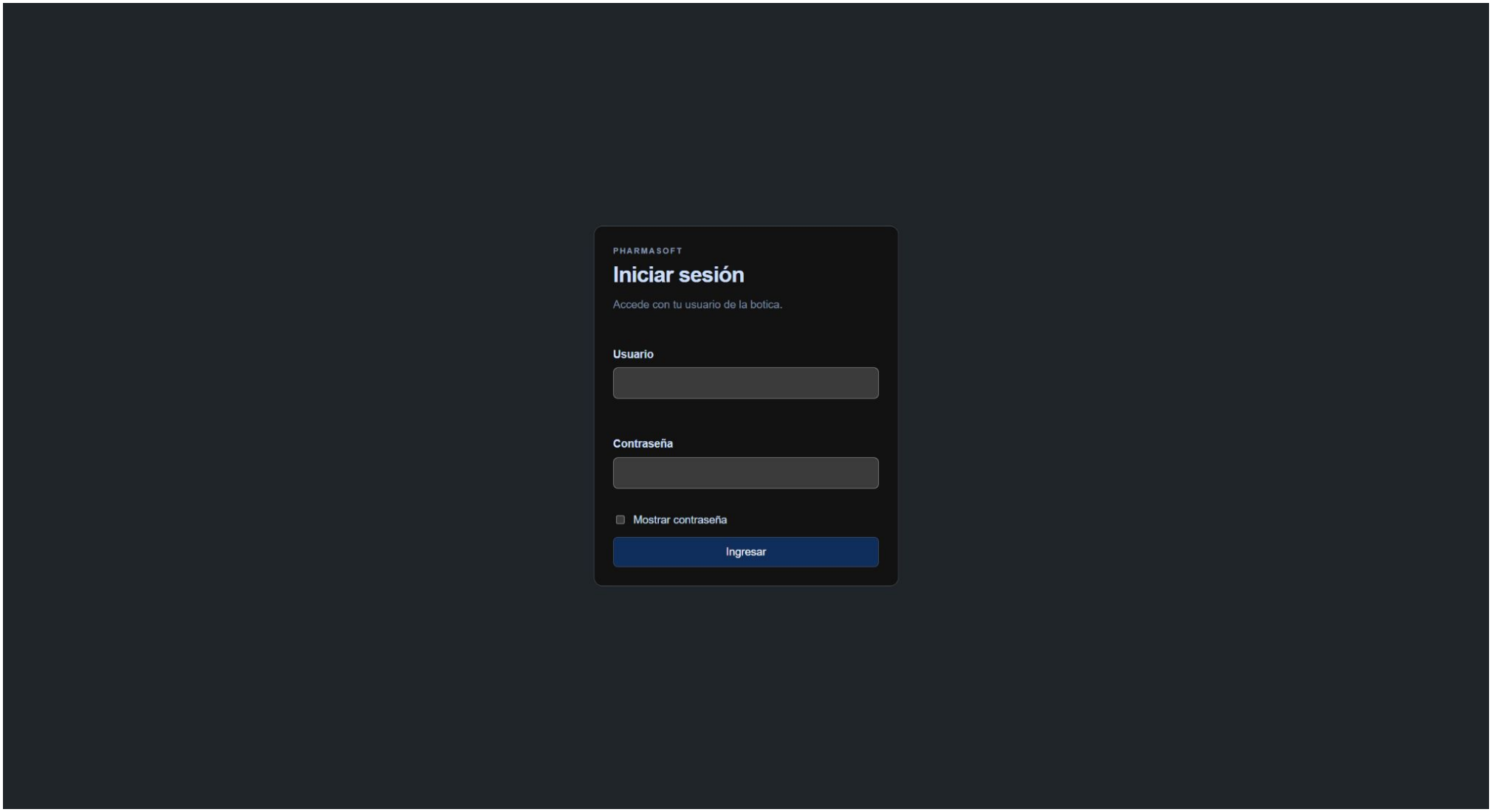
Resultado: La URL directa mostró 403 - Acceso denegado.

PRUEBA 11 / 16 · Login con sesión abierta



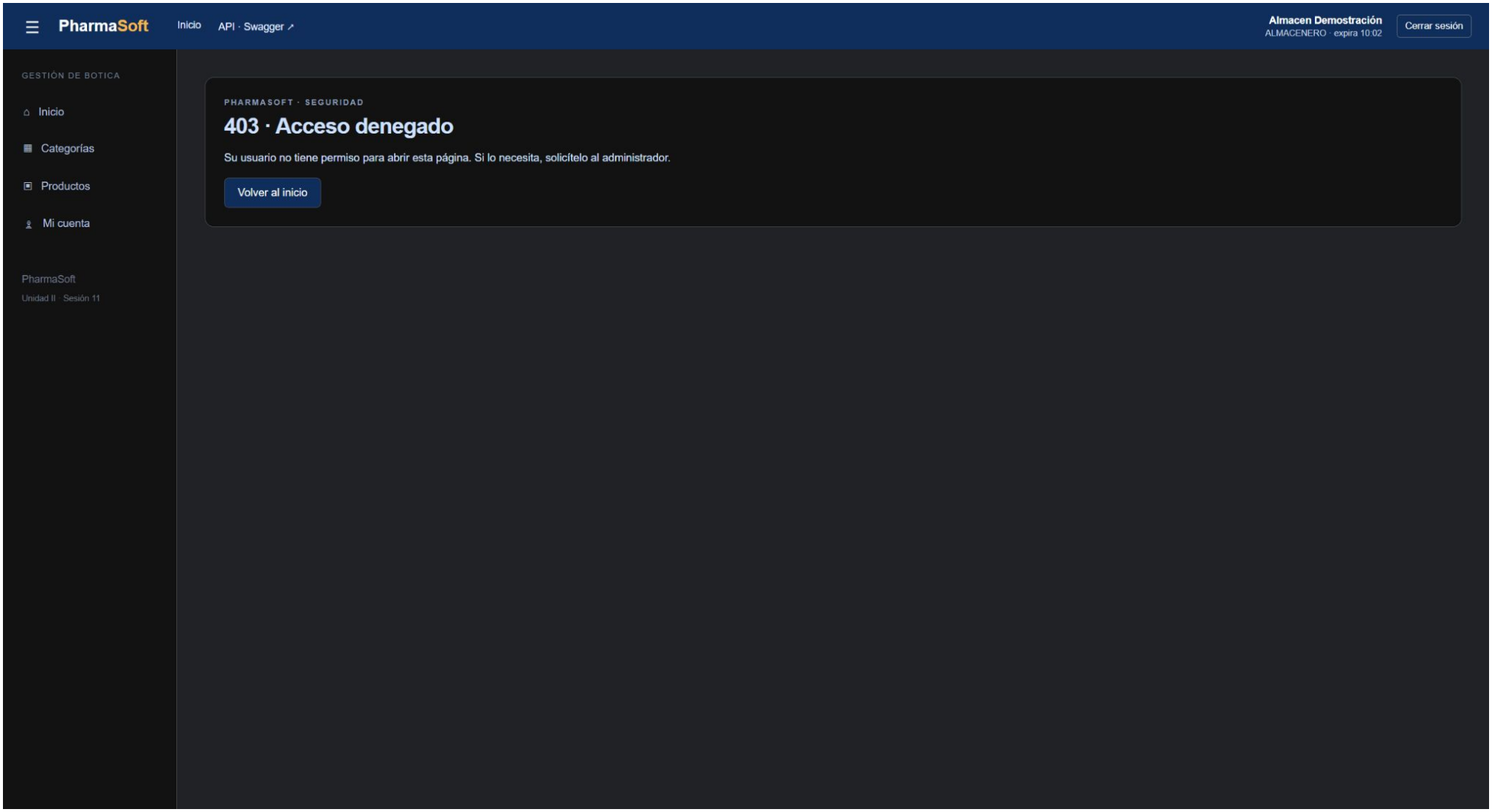
Resultado: La ruta /login redirigió a /inicio.

PRUEBA 12 / 16 · Cerrar sesión



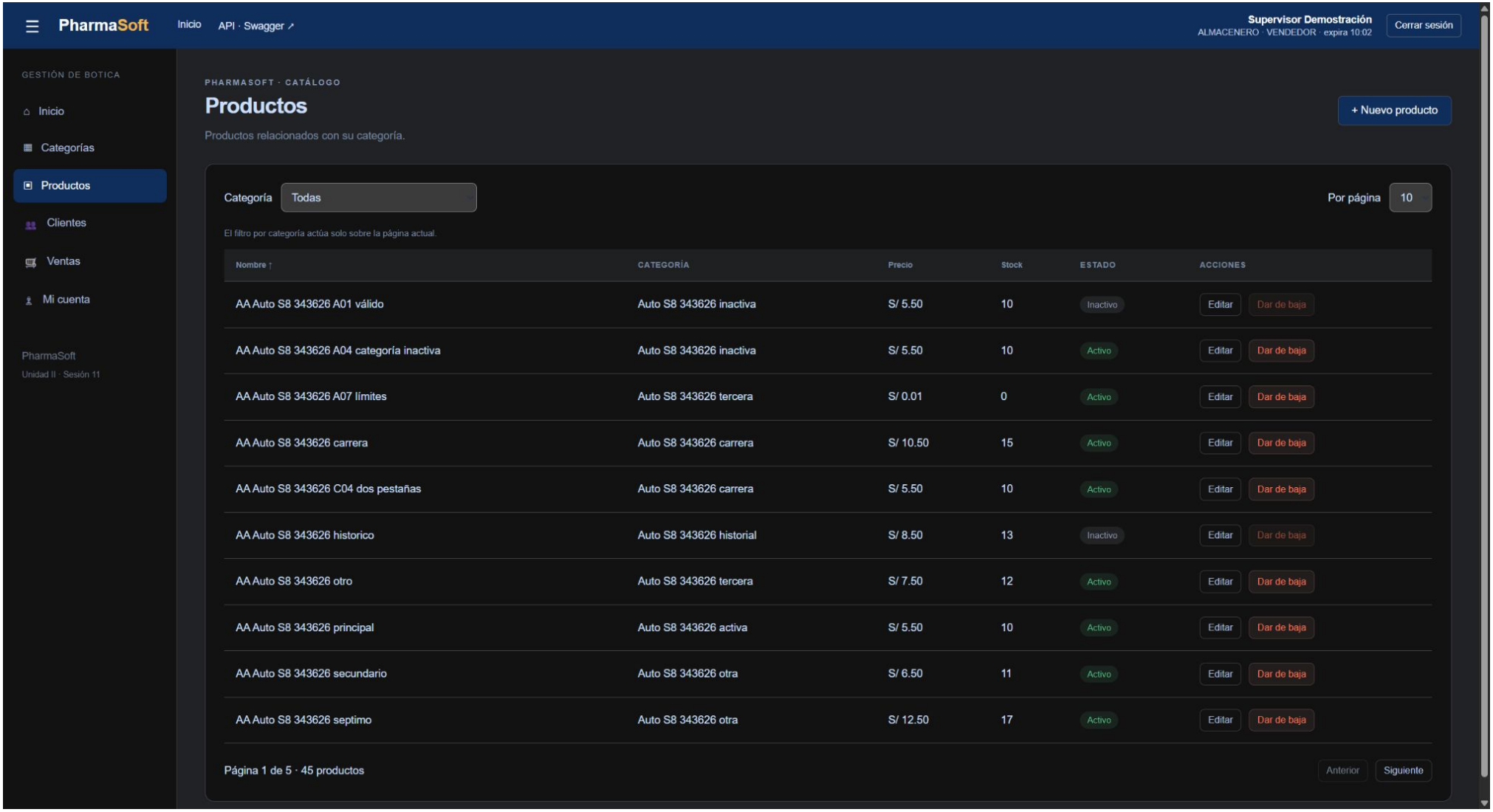
Resultado: Volvió al login; pharmasoft.sesion desapareció de sessionStorage.

PRUEBA 13 / 16 · ALMACENERO en /ventas



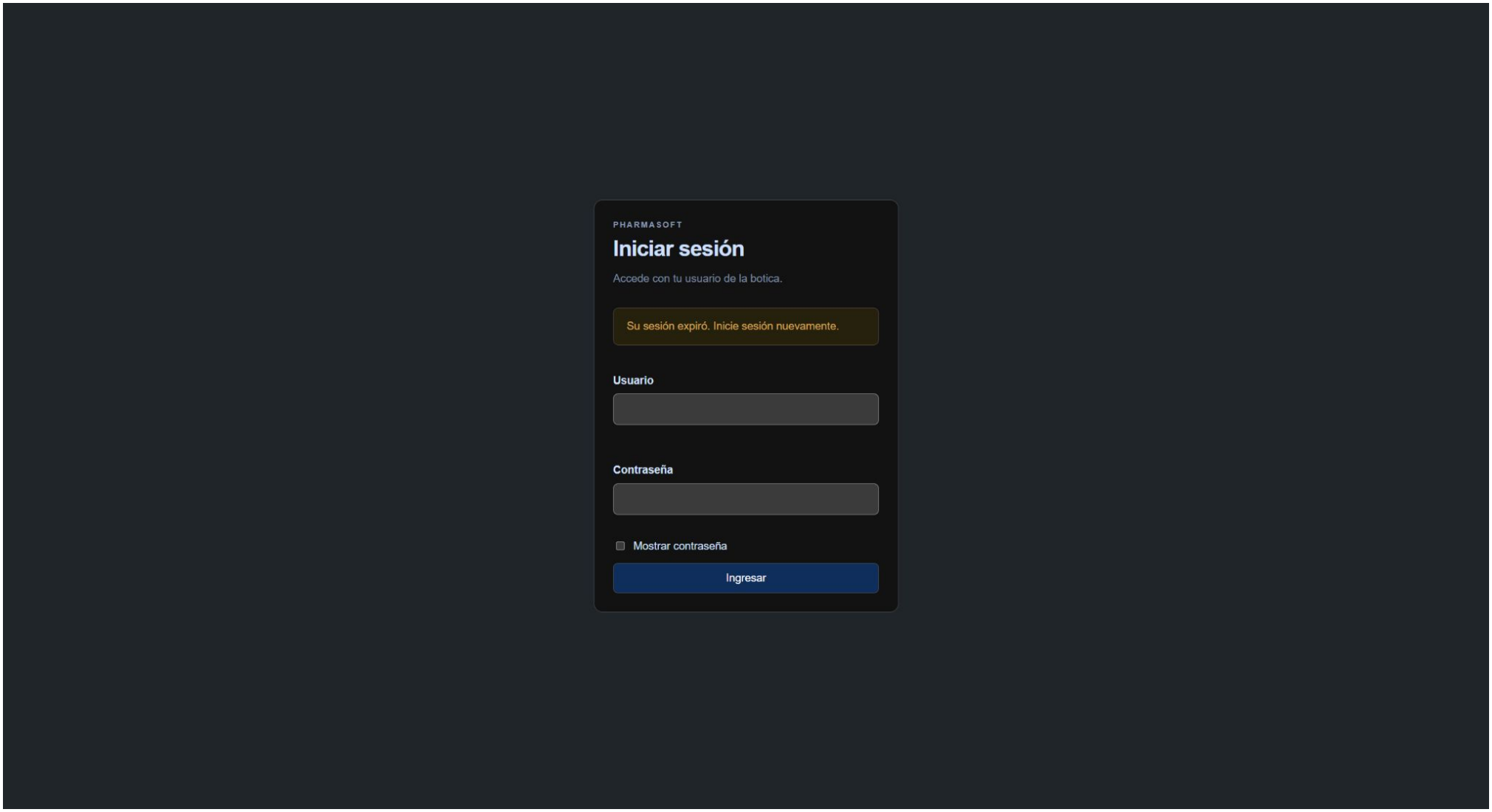
Resultado: Ventas no apareció en el menú y la URL directa mostró acceso denegado.

PRUEBA 14 / 16 · SUPERVISOR con permisos unidos



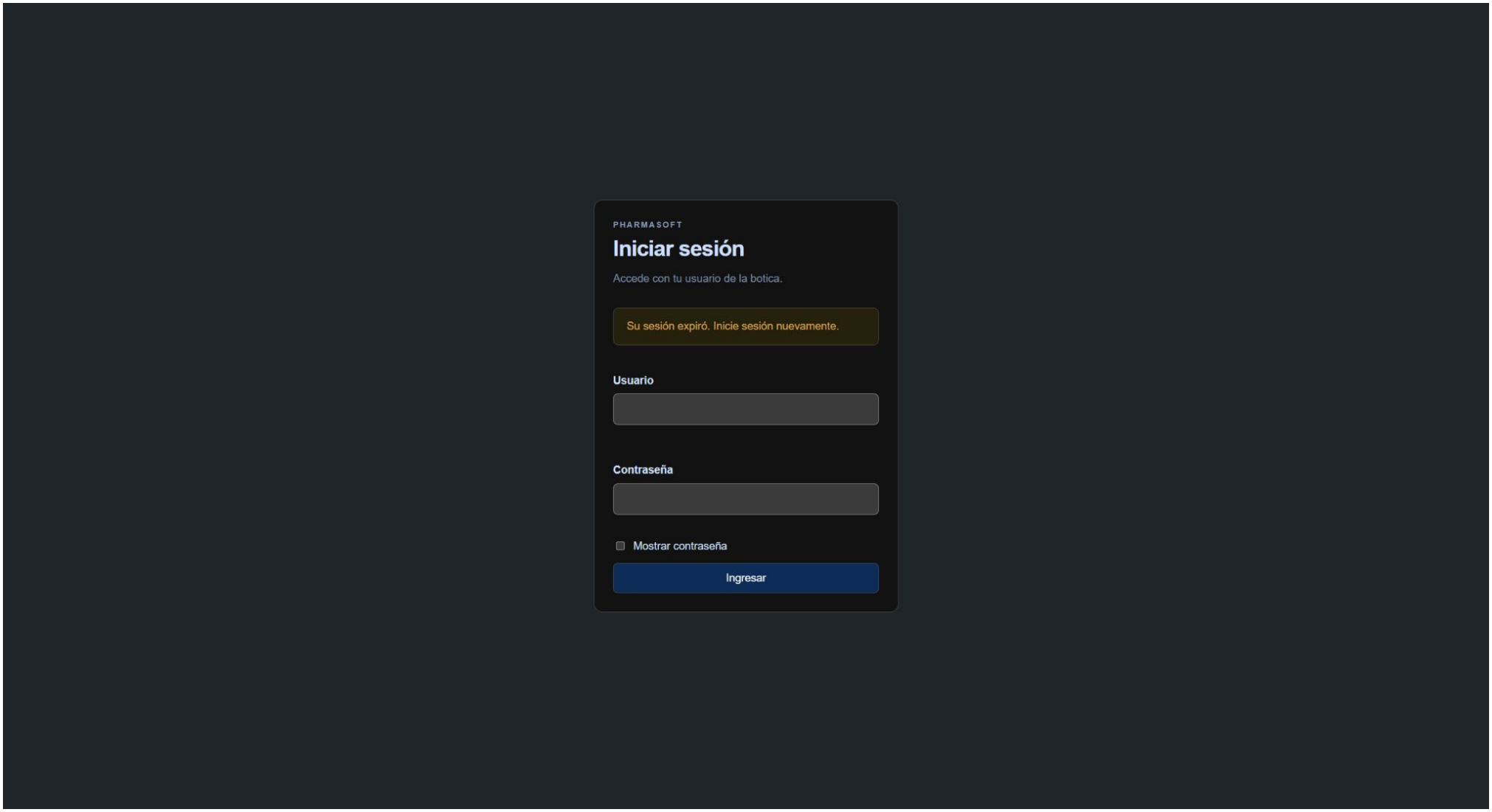
Resultado: Mostró Ventas y las acciones Nuevo producto, Editar y Dar de baja.

PRUEBA 15 / 16 · Token modificado y F5



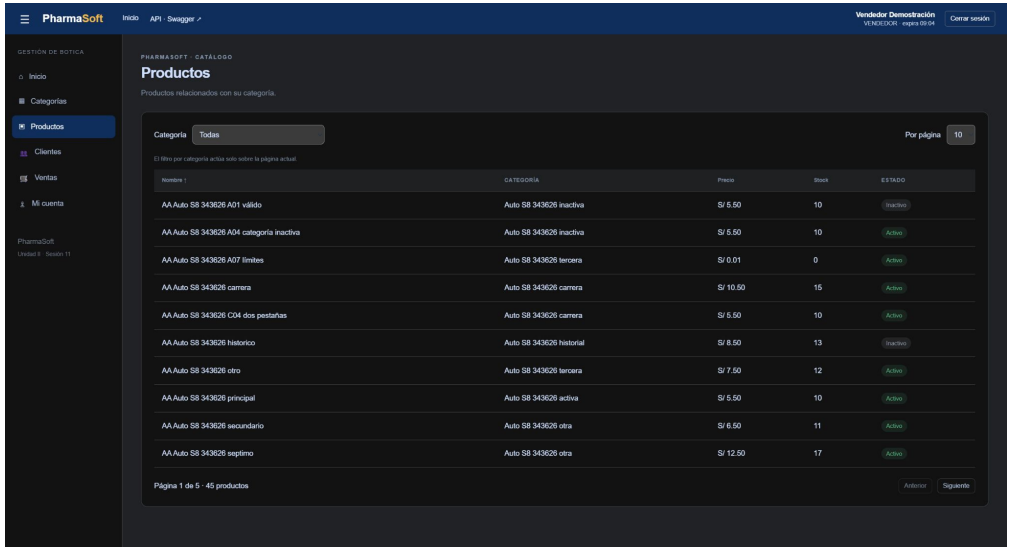
Resultado: La SPA cerró la sesión y mostró Su sesión expiró con retorno a /productos.

PRUEBA 16 / 16 · Expiración automática a un minuto

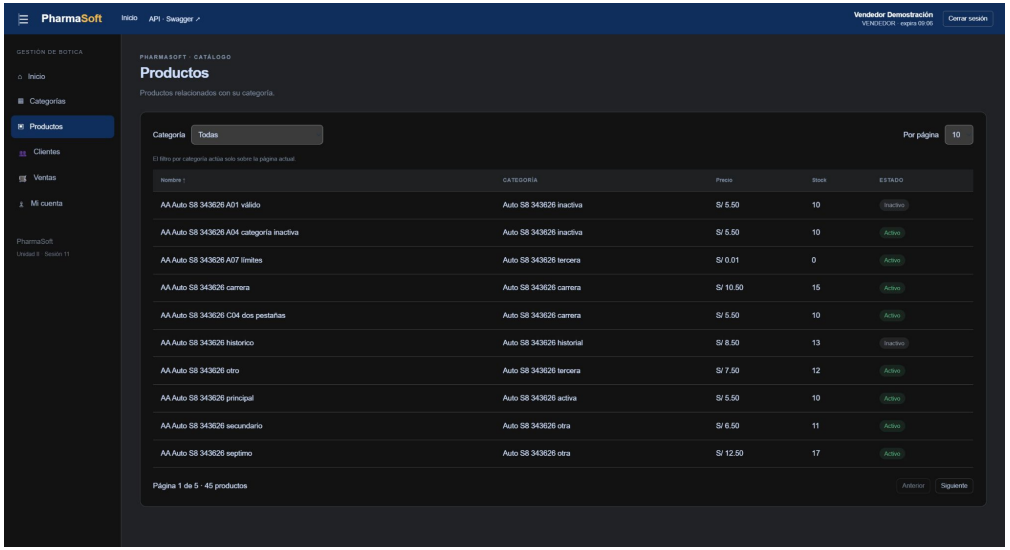


Resultado: Al minuto volvió al login con aviso y returnUrl=/productos; después regresó a Productos.

PRUEBA 16 / 16 · Inicio y retorno tras el vencimiento



Antes: sesión en Productos con JWT de 1 minuto.



Después: el nuevo ingreso regresa a Productos.

La vigencia temporal de un minuto se retiró al terminar la prueba. Las cinco contraseñas originales de demostración fueron restauradas y comprobadas.

CAPTURA DEL CÓDIGO · AuthService: signal, login y sessionStorage

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly http = inject(HttpClient);
  private readonly router = inject(Router);
  private readonly state = signal<Session | null>(this.read());
  private expirationTimer?: ReturnType<typeof setTimeout>;

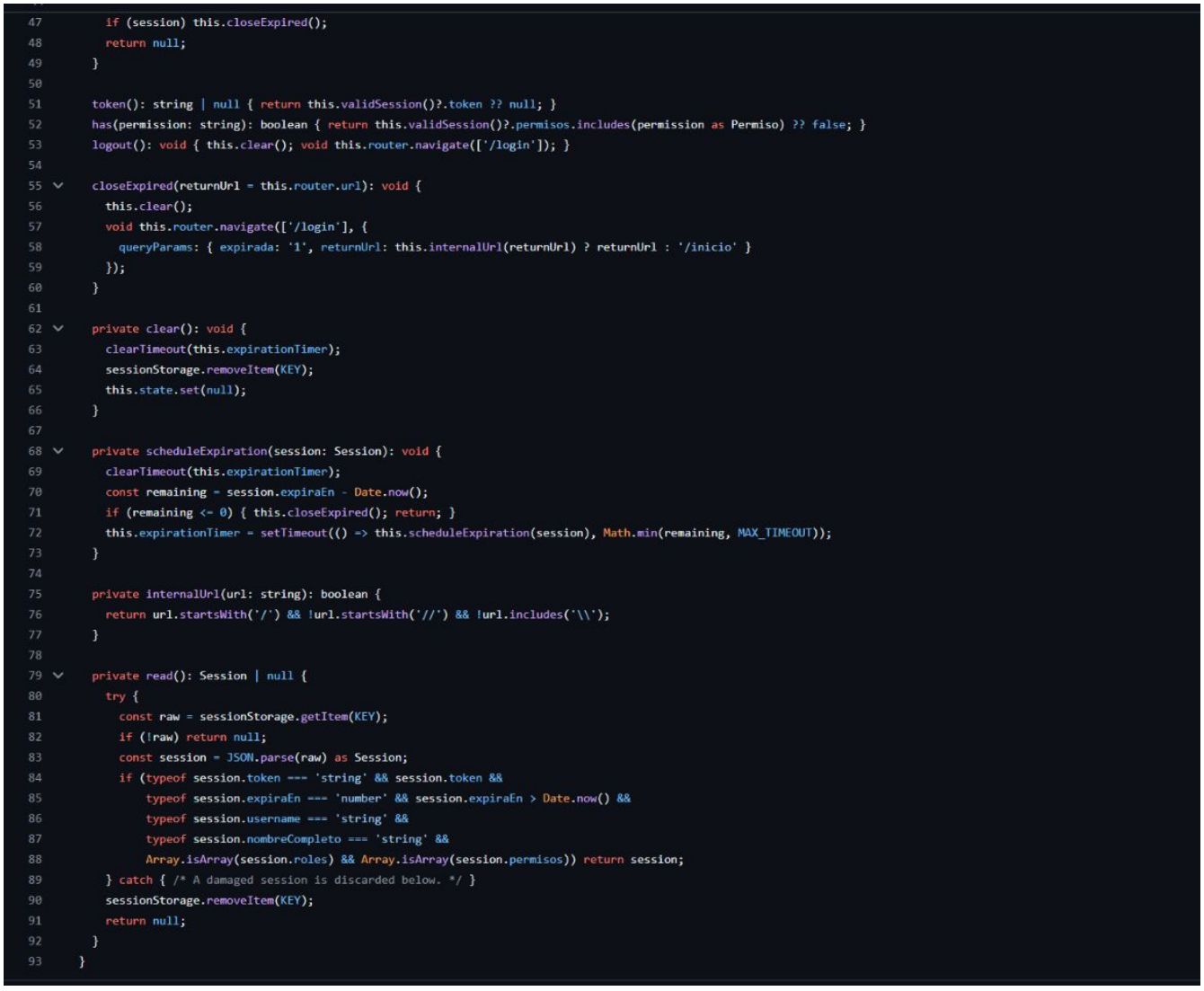
  readonly session = this.state.asReadonly();
  readonly authenticated = computed(() => this.state() !== null);

  constructor() {
    const restored = this.state();
    if (restored) this.scheduleExpiration(restored);
  }

  login(username: string, password: string) {
    return this.http.post<LoginResponse>(`${environment.apiUrl}/auth/login`, { username, password }).pipe(
      map(response => ({
        token: response.token,
        username: response.username,
        nombreCompleto: response.nombreCompleto,
        roles: response.roles,
        permisos: response.permisos,
        expiraEn: Date.now() + response.expiresIn * 1000
      }) satisfies Session),
      tap(session => {
        sessionStorage.setItem(KEY, JSON.stringify(session));
        this.state.set(session);
        this.scheduleExpiration(session);
      })
    );
  }
```

Fuente: rama feature/seguridad-frontend-tinoco publicada en GitHub. Captura recortada para lectura.

CAPTURA DEL CÓDIGO · AuthService: expiración y recuperación



```
47        if (session) this.closeExpired();
48        return null;
49      }
50
51      token(): string | null { return this.validSession()?.token ?? null; }
52      has(permission: string): boolean { return this.validSession()?.permisos.includes(permission as Permiso) ?? false; }
53      logout(): void { this.clear(); void this.router.navigate(['/login']); }
54
55      closeExpired(returnUrl = this.router.url): void {
56        this.clear();
57        void this.router.navigate(['/login'], {
58          queryParams: { expirada: '1', returnUrl: this.internalUrl(returnUrl) ? returnUrl : '/inicio' }
59        });
60      }
61
62      private clear(): void {
63        clearTimeout(this.expirationTimer);
64        sessionStorage.removeItem(KEY);
65        this.state.set(null);
66      }
67
68      private scheduleExpiration(session: Session): void {
69        clearTimeout(this.expirationTimer);
70        const remaining = session.expiraEn - Date.now();
71        if (remaining <= 0) { this.closeExpired(); return; }
72        this.expirationTimer = setTimeout(() => this.scheduleExpiration(session), Math.min(remaining, MAX_TIMEOUT));
73      }
74
75      private internalUrl(url: string): boolean {
76        return url.startsWith('/') && !url.startsWith('//') && !url.includes('\\');
77      }
78
79      private read(): Session | null {
80        try {
81          const raw = sessionStorage.getItem(KEY);
82          if (!raw) return null;
83          const session = JSON.parse(raw) as Session;
84          if (typeof session.token === 'string' && session.token &&
85              typeof session.expiraEn === 'number' && session.expiraEn > Date.now() &&
86              typeof session.username === 'string' &&
87              typeof session.nombreCompleto === 'string' &&
88              Array.isArray(session.roles) && Array.isArray(session.permisos)) return session;
89        } catch { /* A damaged session is discarded below. */ }
90        sessionStorage.removeItem(KEY);
91        return null;
92      }
93    }
```

Fuente: rama feature/seguridad-frontend-tinoco publicada en GitHub. Captura recortada para lectura.

CAPTURA DEL CÓDIGO · Interceptores de token y error

Code Blame 28 lines (25 loc) · 1.26 KB

```
1   import { HttpErrorResponse, HttpInterceptorFn } from '@angular/common/http';
2   import { inject } from '@angular/core';
3   import { Router } from '@angular/router';
4   import { catchError, throwError } from 'rxjs';
5   import { environment } from '../../../environments/environment';
6   import { AuthService } from './auth.service';
7
8   const apiUrl = environment.apiUrl.replace(/\/$/, '');
9   const isApiRequest = (url: string) => url === apiUrl || url.startsWith(`${apiUrl}/`);
10
11  export const authInterceptor: HttpInterceptorFn = (request, next) => {
12    const token = inject(AuthService).token();
13    if (!token || !isApiRequest(request.url) || request.url === `${apiUrl}/auth/login`) return next(request);
14    return next(request.clone({ setHeaders: { Authorization: `Bearer ${token}` } }));
15  };
16
17  export const errorInterceptor: HttpInterceptorFn = (request, next) => {
18    const auth = inject(AuthService);
19    const router = inject(Router);
20    return next(request).pipe(catchError((error: HttpErrorResponse) => {
21      if (isApiRequest(request.url) && error.status === 401 &&
22          request.url !== `${apiUrl}/auth/login` && auth.authenticated()) {
23        auth.closeExpired(router.url);
24      }
25      // A 403 leaves the session intact so the current page can show the API message.
26      return throwError(() => error);
27    }));
28  };
```

Fuente: rama feature/seguridad-frontend-tinoco publicada en GitHub. Captura recortada para lectura.

CAPTURA DEL CÓDIGO · Guards de sesión, invitado y permiso

Code Blame 19 lines (16 loc) · 779 Bytes

```
1   import { inject } from '@angular/core';
2   import { CanActivateFn, Router } from '@angular/router';
3   import { Permiso } from './auth.model';
4   import { AuthService } from './auth.service';
5
6   export const authGuard: CanActivateFn = (_route, state) =>
7     inject(AuthService).validSession()
8       ? true
9       : inject(Router).createUrlTree(['/login'], { queryParams: { returnUrl: state.url } });
10
11  export const guestGuard: CanActivateFn = () =>
12    inject(AuthService).validSession() ? inject(Router).createUrlTree(['/inicio']) : true;
13
14  export const permissionGuard: CanActivateFn = route => {
15    const permission = route.data['permission'] as Permiso | undefined;
16    return !permission || inject(AuthService).has(permission)
17      ? true
18      : inject(Router).createUrlTree(['/acceso-denegado']);
19  };
```

Fuente: rama feature/seguridad-frontend-tinoco publicada en GitHub. Captura recortada para lectura.

CAPTURA DEL CÓDIGO · Rutas protegidas

Code Blame 23 lines (22 loc) · 2.1 KB Raw

```
1   import { Routes } from '@angular/router';
2   import { MainLayout } from './layout/main-layout/main-layout';
3   import { authGuard, guestGuard, permissionGuard } from './core/auth/auth.guard';
4
5   export const routes: Routes = [
6     { path: 'login', canActivate: [guestGuard], loadComponent: () => import('./features/login/login').then(m => m.Login) },
7     {
8       path: '', component: MainLayout, canActivate: [authGuard], canActivateChild: [authGuard],
9       children: [
10        { path: '', pathMatch: 'full', redirectTo: 'inicio' },
11        { path: 'inicio', loadComponent: () => import('./features/inicio/inicio').then(m => m.Inicio) },
12        { path: 'acceso-denegado', loadComponent: () => import('./shared/pages/acceso-denegado/acceso-denegado').then(m => m.AccesoDenegado) },
13        { path: 'clientes', canActivate: [permissionGuard], data: {permission: 'CLIENTES_LEER'}, loadChildren: () => import('./features/clientes/clientes.routes').then(m => m.
14        { path: 'productos', canActivate: [permissionGuard], data: {permission: 'CATALOGO_LEER'}, loadChildren: () => import('./features/productos/productos.routes').then(m =>
15        { path: 'ventas', canActivate: [permissionGuard], data: {permission: 'VENTAS_LEER'}, loadChildren: () => import('./features/ventas/ventas.routes').then(m => m.VENTAS_F
16        { path: 'reportes', canActivate: [permissionGuard], data: {permission: 'REPORTES_VER'}, loadChildren: () => import('./features/reportes/reportes.routes').then(m => m.F
17        { path: 'categorias', canActivate: [permissionGuard], data: {permission: 'CATALOGO_LEER'}, loadChildren: () => import('./features/categorias/categorias.routes').then(m
18        { path: 'usuarios', canActivate: [permissionGuard], data: {permission: 'USUARIOS_ADMINISTRAR'}, loadComponent: () => import('./features/usuarios/usuarios').then(m => m
19        { path: 'cuenta', loadComponent: () => import('./features/cuenta/cuenta').then(m => m.Cuenta) }
20      ]
21    },
22    { path: '**', loadComponent: () => import('./shared/pages/no-encontrado/no-encontrado').then(m => m.NoEncontrado) }
23  ];
```

Fuente: rama feature/seguridad-frontend-tinoco publicada en GitHub. Captura recortada para lectura.

CAPTURA DEL CÓDIGO · Directiva estructural de permisos

```typescript
import { Directive, effect, inject, input, TemplateRef, ViewContainerRef } from '@angular/core';
import { Permiso } from '../../core/auth/auth.model';
import { AuthService } from '../../core/auth/auth.service';

@Directive({ selector: '[appSiPermiso]' })
export class SiPermiso {
  private readonly auth = inject(AuthService);
  private readonly template = inject(TemplateRef<unknown>);
  private readonly container = inject(ViewContainerRef);
  private visible = false;
  readonly appSiPermiso = input.required<Permiso>();

  constructor() {
    effect(() => {
      const allowed = this.auth.has(this.appSiPermiso());
      if (allowed && !this.visible) {
        this.container.createEmbeddedView(this.template);
        this.visible = true;
      } else if (!allowed && this.visible) {
        this.container.clear();
        this.visible = false;
      }
    });
  }
}
```

Fuente: rama feature/seguridad-frontend-tinoco publicada en GitHub. Captura recortada para lectura.

Código: https://github.com/7inox/pharmacia-ap9/tree/feature/seguridad-frontend-tinoco

La rama incluye seis commits de seguridad frontend y datos de demostración. El login consume el campo real expiresIn del backend y almacena expiraEn en la sesión.

Comprobaciones: npm run build, npm test -- --watch=false, compilación Maven, conexión SYSVENTAS a FREEPDB1 y las 16 pruebas de interfaz. La inspección de Red confirmó que el login no envía Bearer y GET /productos sí lo envía. Los valores de JWT y las contraseñas no aparecen en este documento.

Material de la sesión

Guía práctica de la sesión 11, Facultad de Ingeniería y Arquitectura, EP Ingeniería de Sistemas, Lenguaje de Programación II.

Entrega: Tinoco_LP2_S11_Practica.pdf